Manual del Programador: Mini Juego Piedra, Papel o Tijera con Persistencia

Malek Cheheid

E.E.S.T. N° 1 "Eduardo Ader", Vicente López

Laboratorio de Programación

Docente: Yamil Ganduglia

Actividad 7

16 de octubre de 2026

Repositorio: https://github.com/malekcheheid-art/piedrapapelotijera.git

Resumen

Este manual documenta el código fuente del programa en C++ para que otra persona pueda compilarlo, probarlo y modificarlo. Se explican la estructura del proyecto, la estructura de datos Partida, el flujo del programa, la persistencia en archivos de texto, la validación de la entrada, el manejo de errores y las pruebas realizadas. También se incluyen los comandos de compilación y ejecución y el código fuente completo como anexo.

Palabras clave: C++, manual del programador, struct, fstream, ios::app, persistencia.

Introducción

Este documento es la guía técnica del proyecto. El programa es un juego de Piedra, Papel o Tijera que se juega contra la computadora y guarda cada partida en un archivo de texto. El objetivo de este manual es que el código se pueda entender sin necesidad de conocer la actividad de origen, porque cada decisión de diseño está explicada en palabras simples y con fragmentos reales del programa.

Objetivos técnicos

- Explicar cómo compilar y ejecutar el programa.
- Describir la estructura de datos Partida y su utilidad.
- Documentar el flujo de main y de las funciones auxiliares.
- Explicar la escritura y lectura del archivo de historial.
- Detallar las validaciones y el manejo de errores.
- Indicar cómo extender el código y probarlo.

Requisitos técnicos

Para compilar el proyecto se necesita Windows 10 u 11, un compilador g++ con soporte para C++11 y una terminal. Se utilizó TDM-GCC 10.3.0 durante el desarrollo. No se instalan bibliotecas de terceros: el programa solo usa los encabezados estándar iostream, fstream, string, cstdlib y ctime.

Estructura del proyecto

```
src/
├── main.cpp
└── historial_partidas.txt
docs/
├── InformeEEST1_LPR2026_ACT07_G03_Informe_v1.0.0.pdf
├── CHANGELOG.md
└── manuales/
capturas/
├── jugada_consola.png
└── archivo_disco.png
README.md
LICENSE
.gitignore
.gitattributes
```

El código fuente está entero en src/main.cpp. El archivo historial_partidas.txt no se escribe a mano: lo genera el propio programa la primera vez que se juega.

Compilación y ejecución

Desde la raíz del proyecto se compila con el siguiente comando de g++:

```
g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe
```

La opción -std=c++11 indica el estándar del lenguaje. Si la compilación es correcta, no aparece ningún mensaje y se crea el ejecutable dentro de la carpeta src. Para ejecutarlo se escribe:

```
.\src\piedrapapelotijera.exe
```

El ejecutable debe iniciarse desde la raíz del repositorio porque el archivo de historial se guarda en la ruta src/historial_partidas.txt. Si se ejecuta desde otra carpeta, el programa puede avisar que no encuentra el archivo.

Encabezados y constantes

El archivo comienza con los encabezados estándar y una constante que define la ruta del historial:

```
#include <iostream>
#include <fstream>
#include <string>
#include <cstdlib>
#include <ctime>

const string ARCHIVO_HISTORIAL = "src/historial_partidas.txt";
```

Usar una constante para la ruta evita repetir el texto en varias funciones y permite cambiarla en un solo lugar si la estructura del proyecto cambia.

Estructura Partida

```
struct Partida {
   int idPartida;
   string jugador;
   string eleccionJugador;
   string eleccionIA;
   string resultado;
};
```

Partida agrupa datos de tipos distintos que pertenecen a la misma entidad. Esto evita pasar cinco parámetros sueltos a cada función y hace que el historial se pueda leer y guardar de forma uniforme.

Función main y flujo del programa

La función main inicializa la semilla aleatoria, carga el contador de identificadores y entra en un bucle do while que muestra el menú principal hasta que el usuario elige la opción 3.

```
srand(time(0));
int contadorID = cargarContadorID();
int opcion = 0;

do {
   // se muestra el menú
   cin >> opcion;
   if (opcion == 1) {
      jugarPartida(contadorID);
   } else if (opcion == 2) {
      leerHistorialDisco();
   }
} while (opcion != 3);
```

Si la entrada no es numérica, se limpia el estado de cin con cin.clear() e ignore, de modo que el bucle no se bloquee. El menú ofrece tres acciones: jugar una partida, ver el historial guardado y salir.

Función jugarPartida

Esta función recibe el contador por referencia para poder incrementarlo. Primero asigna el identificador a la partida, luego pide el nombre del jugador y su elección, genera la jugada de la computadora y evalúa las reglas.

```
Partida p;
p.idPartida = contadorID++;

cin.ignore();
getline(cin, p.jugador);
```

```
int opcIA = (rand() % 3) + 1;
```

Antes de usar getline se llama a cin.ignore() porque el operador cin deja el carácter Enter pendiente en el búfer. Sin esa limpieza, el nombre se leería vacío. El nombre se vuelve a pedir mientras siga vacío y la jugada se vuelve a pedir mientras no sea 1, 2 o 3.

La comparación entre cadenas decide el resultado: si ambas elecciones son iguales es Empate; si el jugador elige Piedra contra Tijera, Papel contra Piedra o Tijera contra Papel es Victoria; en cualquier otro caso es Derrota.

Función guardarEnDisco

La escritura se hace con ofstream en modo ios::app. Ese modo agrega las líneas al final del archivo y evita borrar las partidas anteriores. Los datos se escriben separados por comas, siguiendo un formato CSV sencillo.

```
ofstream archivo(ARCHIVO_HISTORIAL, ios::app);
if (!archivo.is_open()) {
   cerr << "[ERROR] No se pudo abrir el archivo." << endl;
   return;
}
archivo << p.idPartida << ","
       << p.jugador << ","
       << p.eleccionJugador << ","
       << p.eleccionIA << ","
       << p.resultado << endl;
archivo.close();
```

Función leerHistorialDisco

La lectura se hace con ifstream y cinco llamadas a getline con delimitador coma. El bucle se detiene cuando ya no quedan registros completos.

```
while (getline(archivo, idStr, ',') &&
       getline(archivo, jugador, ',') &&
       getline(archivo, elJug, ',') &&
       getline(archivo, elIA, ',') &&
       getline(archivo, resultado)) {
    total++;
}
```

Si el archivo no existe, el programa informa que todavía no hay partidas guardadas. Si existe pero está vacío, avisa que el historial está vacío. En ambos casos la ejecución continúa sin errores.

Función cargarContadorID

Para que los identificadores no se repitan entre ejecuciones, esta función abre el archivo, recorre los registros y se queda con el ID más alto. Luego devuelve ese número más uno. Si el archivo todavía no existe, devuelve 1.

```
int cargarContadorID() {
    ifstream archivo(ARCHIVO_HISTORIAL);
    if (!archivo.is_open()) return 1;
    int maxId = 0;
    // se leen los registros y se conserva el ID mayor
    return maxId + 1;
}
```

Validación de la entrada

La validación se aplica en dos lugares: el menú principal y la elección de jugada. Si el usuario escribe una letra en lugar de un número, se usa cin.clear() para limpiar el error y cin.ignore() para descartar el resto de la línea. El nombre del jugador tampoco puede quedar vacío.

Pruebas realizadas

Para comprobar la persistencia se jugaron dos partidas. En la primera, el jugador Malek eligió Piedra y la computadora también eligió Piedra, con resultado Empate. En la segunda, el jugador malek eligió Papel y la computadora eligió Piedra, con resultado Victoria. El archivo historial_partidas.txt mostró las dos líneas en formato CSV y el historial se recuperó al volver a abrir el programa.

Depuración de problemas frecuentes

- Si g++ no se reconoce, hay que instalar el compilador o usar un entorno en línea.
- Si el ejecutable no encuentra el historial, se debe iniciar desde la raíz del proyecto.
- Si se saltea la pregunta del nombre, falta cin.ignore() antes de getline.
- Si las partidas anteriores desaparecen, se está usando ofstream sin ios::app.
- Si el programa se queda en un bucle, hay que limpiar cin con cin.clear() e ignore.

Cómo extender el programa

Para agregar un campo nuevo, por ejemplo la fecha de la partida, hay que modificar la estructura Partida, la escritura en guardarEnDisco y la lectura en leerHistorialDisco. Si se agrega un campo al final de la línea, los registros viejos siguen siendo compatibles porque siguen teniendo cinco comas.

Otra mejora posible es mostrar un marcador de victorias. Para eso se puede agregar un contador que se lea del archivo al iniciar y que se incremente cada vez que el resultado sea Victoria. La técnica es la misma que se usa en el Proyecto Integrador Anual para registrar eventos de hardware o logs de usuarios en un archivo de texto.

Convenciones de código

Las funciones usan minúsculas y describen su propósito: jugarPartida, guardarEnDisco, leerHistorialDisco y cargarContadorID. Los mensajes se escriben entre corchetes para identificarlos en la consola, por ejemplo [DISCO] o [SISTEMA]. El código usa indentación de cuatro espacios y coloca una llave de apertura y cierre por línea.

Referencias

GeeksforGeeks. (s. f.). *Structs y estructuras de datos en C++*. GeeksforGeeks. https://www.geeksforgeeks.org/cpp/structures-in-cpp/

GeeksforGeeks. (s. f.). *cin.ignore() Function in C++*. GeeksforGeeks. https://www.geeksforgeeks.org/cpp/cin-ignore-function-in-cpp/

cppreference.com. (s. f.). *std::basic_ofstream::open*. En cppreference.com. https://en.cppreference.com/w/cpp/io/basic_ofstream/open

Anexo A. Código fuente completo

```
/*
  ================================================================

  Materia: Laboratorio de Programacion (LPR) -- 5to Ano
  Institucion: E.E.S.T. N° 1 "Eduardo Ader" -- Vicente Lopez
  Archivo: src/main.cpp
  Actividad 7: Mini Juego Piedra, Papel o Tijera + Persistencia en Disco

  Autor: Malek Cheheid  (Grupo A)
  Docente: Yamil Ganduglia

  ================================================================
*/

#include <iostream>
#include <fstream>
```

```
#include <string>
#include <cstdlib>
#include <ctime>

using namespace std;

const string ARCHIVO_HISTORIAL = "src/historial_partidas.txt";

struct Partida {
    int idPartida;
    string jugador;
    string eleccionJugador;
    string eleccionIA;
    string resultado;
};

void jugarPartida(int& contadorID);
void guardarEnDisco(const Partida& p);
void leerHistorialDisco();
int cargarContadorID();

int main() {
    srand(time(0));

    int contadorID = cargarContadorID();
    int opcion = 0;

    cout << "=======================================================" << endl;
    cout << " LPR 2026 - ACTIVIDAD 7: PIEDRA, PAPEL O TIJERA " << endl;
    cout << " Estudiante / Grupo: Malek Cheheid (Grupo A) " << endl;
    cout << "=======================================================" << endl;

    do {
        cout << "\n--- MENU PRINCIPAL DE JUEGO Y PERSISTENCIA ---" << endl;
        cout << "1. Jugar Partida (Piedra, Papel o Tijera)" << endl;
        cout << "2. Ver Historial de Partidas Guardadas en Disco (.txt)" << endl;
        cout << "3. Salir del Sistema" << endl;
        cout << "Ingrese una opcion (1-3): ";
        if (!(cin >> opcion)) {
```

```
            cin.clear();
            cin.ignore(10000, '\n');
            opcion = 0;
        }

        if (opcion == 1) {
            jugarPartida(contadorID);
        } else if (opcion == 2) {
            leerHistorialDisco();
        } else if (opcion == 3) {
            cout << "\n[SISTEMA] Saliendo del programa... Datos resguardados en disco." << endl;
        } else {
            cout << "\n[ERROR] Opcion invalida. Reintente." << endl;
        }
    } while (opcion != 3);

    return 0;
}

void jugarPartida(int& contadorID) {
    Partida p;
    p.idPartida = contadorID++;

    cout << "\n--- NUEVA PARTIDA #" << p.idPartida << " ---" << endl;

    cin.ignore(); // Limpieza de bufer
    do {
        cout << "Ingrese su Nombre / Nickname: ";
        getline(cin, p.jugador);
        if (p.jugador.empty()) {
            cout << "[ERROR] El nombre no puede estar vacio." << endl;
        }
    } while (p.jugador.empty());

    int opcJugador = 0;
    do {
        cout << "\nElija su jugada:" << endl;
        cout << "1. Piedra" << endl;
        cout << "2. Papel" << endl;
```

```
        cout << "3. Tijera" << endl;
        cout << "Opcion (1-3): ";
        if (!(cin >> opcJugador)) {
            cin.clear();
            cin.ignore(10000, '\n');
            opcJugador = 0;
        }
        if (opcJugador < 1 || opcJugador > 3) {
            cout << "[ERROR] Opcion invalida. Reintente." << endl;
        }
    } while (opcJugador < 1 || opcJugador > 3);

    if (opcJugador == 1) p.eleccionJugador = "Piedra";
    else if (opcJugador == 2) p.eleccionJugador = "Papel";
    else p.eleccionJugador = "Tijera";

    int opcIA = (rand() % 3) + 1;
    if (opcIA == 1) p.eleccionIA = "Piedra";
    else if (opcIA == 2) p.eleccionIA = "Papel";
    else p.eleccionIA = "Tijera";

    cout << "\n[JUGADA] " << p.jugador << " eligio: " << p.eleccionJugador << endl;
    cout << "[JUGADA] La IA eligio: " << p.eleccionIA << endl;

    if (p.eleccionJugador == p.eleccionIA) {
        p.resultado = "Empate";
    } else if ((p.eleccionJugador == "Piedra" && p.eleccionIA == "Tijera") ||
               (p.eleccionJugador == "Papel" && p.eleccionIA == "Piedra") ||
               (p.eleccionJugador == "Tijera" && p.eleccionIA == "Papel")) {
        p.resultado = "Victoria";
    } else {
        p.resultado = "Derrota";
    }

    cout << "=> RESULTADO: " << p.resultado << "!" << endl;

    guardarEnDisco(p);
}
```

```
void guardarEnDisco(const Partida& p) {
    ofstream archivo(ARCHIVO_HISTORIAL, ios::app);

    if (!archivo.is_open()) {
        cerr << "[ERROR] No se pudo abrir el archivo para guardar la partida." << endl;
        return;
    }

    archivo << p.idPartida << ","
            << p.jugador << ","
            << p.eleccionJugador << ","
            << p.eleccionIA << ","
            << p.resultado << endl;

    archivo.close();
    cout << "[DISCO] Partida resguardada con exito en 'historial_partidas.txt'." << endl;
}

void leerHistorialDisco() {
    ifstream archivo(ARCHIVO_HISTORIAL);

    if (!archivo.is_open()) {
        cout << "\n[AVISO] Aun no existen partidas guardadas en disco." << endl;
        return;
    }

    cout << "\n=====================================================" << endl;
    cout << " HISTORIAL DE PARTIDAS RECUPERADO DESDE DISCO " << endl;
    cout << "=====================================================" << endl;

    string idStr, jugador, elJug, elIA, resultado;
    int total = 0;

    while (getline(archivo, idStr, ',') &&
           getline(archivo, jugador, ',') &&
           getline(archivo, elJug, ',') &&
           getline(archivo, elIA, ',') &&
           getline(archivo, resultado)) {
        total++;
```

```
        cout << "Partida #" << idStr
             << " | Jugador: " << jugador
             << " | " << elJug << " vs " << elIA
             << " | Resultado: " << resultado << endl;
    }

    archivo.close();

    if (total == 0) {
        cout << "[AVISO] El archivo de historial esta vacio." << endl;
    } else {
        cout << "\n[SISTEMA] Total de registros leidos desde disco: " << total << endl;
    }
    cout << "=========================================================" << endl;
}

int cargarContadorID() {
    ifstream archivo(ARCHIVO_HISTORIAL);
    if (!archivo.is_open()) return 1;

    int maxId = 0;
    string idStr, jugador, elJug, elIA, resultado;
    while (getline(archivo, idStr, ',') &&
           getline(archivo, jugador, ',') &&
           getline(archivo, elJug, ',') &&
           getline(archivo, elIA, ',') &&
           getline(archivo, resultado)) {
        try {
            int id = stoi(idStr);
            if (id > maxId) maxId = id;
        } catch (...) {
            // Ignorar lineas mal formadas
        }
    }
    archivo.close();
    return maxId + 1;
}
```